LAB W12 · ONE LAB, TWELVE STEPS, EACH ONE BUILDS ON THE LAST

The coffee bar: a menu editor that remembers

The Corner Store owner wants one screen to manage the coffees on the menu: see them all, add a new one, change one, delete one, and search them. The pages are built. The routes already check every request. Tonight you connect it all to the real database from Tuesday, the way Wednesday's demo did, so that every change is still there tomorrow.

This lab is meant to stretch you. Every step tells you what to build and how to check it. If you get stuck, every step also links to the full code for that step in the Code reference at the back. Try it yourself first, then look.

Contents

0. Run it and meet the starting point
1. Install Drizzle and drizzle-kit
2. The connection string in `.env.local`
3. `drizzle.config.ts` and a script
4. Pull: let drizzle-kit write the schema
5. The Neon provider
6. The item service, and the list of coffees
7. One coffee
8. Create: a new coffee
9. Edit: change a coffee
10. Delete, and Tuesday's foreign key
11. Search by name: `?search=`
12. A second parameter: `?maxPrice=`

What "done" looks like · Going further · Things that will bite you · Code reference

What you'll have at the end

ADDRESS	VERB	CALLS THE SERVICE'S	WHAT IT DOES
`/api/coffees`	GET	`getCoffees`	Every coffee, or the ones matching `?search=` and `?maxPrice=`
`/api/coffees`	POST	`createCoffee`	Add a coffee
`/api/coffees/3`	GET	`getCoffee`	One coffee
`/api/coffees/3`	PUT	`updateCoffee`	Change a coffee
`/api/coffees/3`	DELETE	`hasBeenSold`, `deleteCoffee`	Delete a coffee nobody has bought yet

And the files behind them, one job each:

LAYER	FILE	KNOWS	DOESN'T KNOW
Page	`app/page.tsx` and friends (done)	the addresses under `/api/coffees`	how the API gets its answers
Route	`app/api/coffees/.../route.ts`	HTTP: the request, the body, the status code	SQL, Drizzle, the schema
Service	`services/itemService.ts`	every query about items, and the store's rules	HTTP, status codes
Provider	`providers/neon/neonDatabaseProvider.ts`	how to connect to Neon	anything about coffee

USING CLAUDE IN THIS LAB

You have Claude chat open, so use it, but use it to **understand**, not to paste. Start every new chat by pasting this, so it knows exactly what you're working with:

> *"I'm a beginner in a full-stack class. I'm building a Next.js 16 app (App Router, TypeScript). API routes are files at app/api/.../route.ts that export functions named GET, POST, PUT and DELETE. My database is Postgres on Neon. I use Drizzle ORM (drizzle-orm with the neon-http driver from @neondatabase/serverless), and drizzle-kit wrote drizzle/schema.ts by pulling my existing database. My code has three layers: a Neon provider class (providers/neon/neonDatabaseProvider.ts) that is the only file that connects, an ItemService class (services/itemService.ts) that holds every query, and routes that only check the request, call the service and pick a status code. No Prisma, no pg, no SQL strings in the routes, no extra libraries. When I ask for help, explain the idea and give me a hint or a small piece first, not a whole finished file, unless I say I'm completely stuck."*

When something breaks, give it three things: **the file**, **what the terminal printed**, and **what you expected vs. what you saw** (on the page, or the status and body in Postman). "It doesn't work" gets you a guess. Those three get you an answer.

If Claude suggests Prisma, `pg` or `Pool`, `db.query("SELECT ...")`, `drizzle-kit push`, or writing the query straight into the route, it's describing a different setup. Remind it: Drizzle on neon-http, and every query goes in the service.

Never paste your connection string or `.env.local` into a chat. It's the password to your whole database. If Claude needs to see it, replace the part between `:` and `@` with `****` first.

0 Run it and meet the starting point

```
cd mtc-ai-demos/week12/lab1
npm install
npm run dev
```

Keep that terminal where you can see it for the whole lab. When something goes wrong, the page often just says "The API answered 500," and the reason is printed in the terminal.

Open http://localhost:3000. The pages are finished, and you won't change them tonight. Click around:

- The list says **"No coffees found."** Under the search box it shows the address it asked: `/api/coffees`.
- Click **+ New coffee**, type anything, and press **Save**. The form shows **"Not built yet. That's Step 8."** That's the API's answer, a 501, shown on the page.

Now read the two route files. They already do the week 11 part: `POST` and `PUT` read the body inside a `try` / `catch` and check every field, with a 400 for each problem. Where the database belongs, each one has a `TODO` that says which step fills it in:

```
// TODO (Step 8): save it through the service, then answer 201 with the new coffee.
return Response.json({ error: "Not built yet. That's Step 8." }, { status: 501 });
```

The `db/` folder

`db/0001_store.sql` is Tuesday's `store.sql`, the SQL that builds the whole Corner Store database. Real projects keep the SQL that builds their database in a `db/` folder, committed with the code, so anyone can rebuild it. The files are numbered (`0001`, `0002`, ...) so they run in order: each later change to the database is a new numbered file. Tonight there's only one.

You need a database called `store` in your Neon project with this file run in it. If you did Tuesday, you have it. If not, or if you want to start over at any point tonight: Neon → **SQL Editor** → pick `store` in the database dropdown → paste the whole file → **Run**. It takes about a minute, and it puts every table back exactly the way it started.

CHECK IT

DO	EXPECT	YOU SEE
Open `localhost:3000`	**page**	"No coffees found." and "Asked: /api/coffees"
Open `localhost:3000/api/coffees`	**200**	`[]`
Open `localhost:3000/api/coffees/3`	**501**	`{"error": "Not built yet. That's Step 7. (You asked for 3.)"}`

Neon SQL Editor, database `store`:

```
SELECT * FROM items WHERE category_id = 1;
```

8 rows Espresso through Gingerbread Latte. These are the coffees. Gingerbread Latte has `is_available` false.

ASK YOUR CLAUDE

> *"In a web app, what's the difference between the page, the API route, and the database? If my page fetches /api/coffees, which of those three ever talks to the database, and why not the page?"*

1 Install Drizzle and drizzle-kit

Stop the server (`Ctrl` + `C`) and install four packages, in two lines:

```
npm install drizzle-orm @neondatabase/serverless
npm install -D drizzle-kit dotenv
```

- `drizzle-orm` is the part your app uses while it runs. You write `db.select().from(items)`, and it turns that into SQL, sends it, and hands the rows back as JavaScript objects.
- `@neondatabase/serverless` is the driver: how the SQL actually travels to Neon over the internet.
- `drizzle-kit` is the half you run from the terminal. Tonight it does one job: read your database and write the TypeScript description of it for you.
- `dotenv` lets drizzle-kit read `.env.local`. Next reads that file by itself; drizzle-kit is a separate program, so it doesn't.

`-D` means "only needed while building," so those two land under `devDependencies` in `package.json`.

CHECK IT

Open `package.json`. `drizzle-orm` and `@neondatabase/serverless` are under `dependencies`; `drizzle-kit` and `dotenv` are under `devDependencies`.

ASK YOUR CLAUDE

> *"What's an ORM? Explain what drizzle-orm does with a line like db.select().from(items), and why a team might use it instead of writing SQL text by hand."*

> *"What's the difference between dependencies and devDependencies in package.json? Why would drizzle-kit be a dev dependency but drizzle-orm not?"*

2 The connection string in `.env.local`

The connection string is the address **and** the password of your database in one line. It's a secret, so it goes in `.env.local`, which never gets committed. (Same rule as last week's token.)

1. Open `.gitignore` in this folder and find the `.env*.local` line. That's what keeps the file out of Git.
2. Make a new file called `.env.local` next to `package.json`. Make it **in VS Code**, not Notepad, so it doesn't secretly become `.env.local.txt`. Copy the one line from `.env.example` into it.
3. In Neon, open your project and click **Connect**. **Change the database dropdown to** `store` before you copy. It starts on `neondb`, which is the wrong one.
4. Paste the string after the `=`. No quotes, no spaces:

```
NEON_DATABASE_URL=postgresql://neondb_owner:****@ep-something.neon.tech/store?sslmode=require
```

Look at the end of yours: it should say `/store?`, not `/neondb?`.

CHECK IT

Run `git status`. `.env.local` must **not** be listed. If it is, stop and ask: that string is a password, and this repo is public.

ASK YOUR CLAUDE

> *"Here's the shape of a Postgres connection string with the password hidden: postgresql://user:****@host/store?sslmode=require. Explain each part. Which part picks the database, and what does sslmode=require do?"*

3 `drizzle.config.ts` and a script

drizzle-kit needs to know four things, and a file called `drizzle.config.ts` tells it. Make it next to `package.json`:

```
import { defineConfig } from "drizzle-kit";
import dotenv from "dotenv";

dotenv.config({ path: ".env.local" });

export default defineConfig({
  dialect: "postgresql",              // which kind of database
  schema: "./drizzle/schema.ts",      // where our tables are described
  out: "./drizzle",                   // where drizzle-kit writes what it generates
  dbCredentials: { url: process.env.NEON_DATABASE_URL || "" },
});
```

- `dialect`: which kind of database. Postgres.
- `schema`: the file where your tables are described in TypeScript. It doesn't exist yet; the pull in Step 4 writes it.
- `out`: the folder drizzle-kit writes into.
- `dbCredentials.url`: how to reach the database. That only works because of the `dotenv.config(...)` line at the top, which loads `.env.local` into `process.env` first. The `|| ""` just keeps TypeScript happy if it's missing.

Then add one line to the `"scripts"` in `package.json`, so the command has a name you can remember. Don't forget the comma at the end of the line above it:

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "db:pull": "drizzle-kit pull"
},
```

Stuck? Code C1: the config and the script

ASK YOUR CLAUDE

> *"Next.js reads .env.local by itself, so why does my drizzle.config.ts need dotenv.config({ path: '.env.local' })? What would process.env.NEON_DATABASE_URL be without that line?"*

4 Pull: let drizzle-kit write the schema

```
npm run db:pull
```

drizzle-kit connects to your database, reads every table, column, key and rule that `store.sql` made on Tuesday, and writes them out as TypeScript. It never changes the database; pulling only reads.

WHAT YOU SHOULD SEE

```
[✓] 10 tables fetched
[✓] 51 columns fetched
...
[✓] 10 foreign keys fetched
...
[✓] Your SQL migration file ➜ drizzle\0000_<random words>.sql 🚀
[✓] Your schema file is ready ➜ drizzle\schema.ts 🚀
[✓] Your relations file is ready ➜ drizzle\relations.ts 🚀
```

And a new `drizzle/` folder:

FILE	WHAT IT IS
`drizzle/schema.ts`	**The one you use.** Every table, described in TypeScript.
`drizzle/relations.ts`	Which tables point at which. Not needed tonight.
`drizzle/0000_<random words>.sql`	The SQL that would rebuild these tables, commented out. Your `db/0001_store.sql` is the real one.
`drizzle/meta/`	drizzle-kit's own notes. Leave it alone.

`1 TABLES FETCHED`, OR `0`?

Your string points at `neondb`, not `store`. **Delete the whole `drizzle/` folder** (pulling again leaves the wrong first files behind), fix the end of the string in `.env.local`, and pull again. An error about the URL instead? The `dotenv` line is missing from the config, or the file is really called `.env.local.txt`.

4b. Read it, then make one edit

Open `drizzle/schema.ts`, find `export const items`, and put `db/0001_store.sql`'s `CREATE TABLE items` next to it. Every line matches. The name on the left is what **your code** calls the column (`isAvailable`); the one in quotes is its real name in Postgres (`is_available`).

Now the one edit. Postgres sends a `numeric` column back as **text** (`"2.50"`), because turning money into a JavaScript number can lose a fraction of a penny. The page calls `coffee.price.toFixed(2)`, which only works on a number, so tell Drizzle to turn it into one. Change the `price` line in `items`:

```
price: numeric({ precision: 6, scale: 2, mode: "number" }).notNull(),
```

FROM NOW ON, THE FILE IS YOURS

Pulling again rewrites `schema.ts` from scratch, and your edit is gone. Pull once to get started; after that, when the database changes, change the file by hand.

Stuck? Code C2: the items table after the edit. If your pull won't run at all, ask your instructor for `week12/demo1/states/02-pulled-schema/schema.ts`: the same database, so the same file.

ASK YOUR CLAUDE

> *"Here's one table from my Drizzle schema.ts: [paste the items table]. Walk me through each line. What do .notNull(), .default(true) and serial().primaryKey() mean, and which SQL keywords in CREATE TABLE do they match?"*

> *"Why does Postgres send a numeric(6,2) column back as text like "2.50" instead of a number? What could go wrong with money stored as a JavaScript number?"*

5 The Neon provider

A **provider** wraps one thing outside your app, here Neon, and is the **only** file that reads its secret and connects. Everything else asks the provider for the connection. If the connection ever changes, you change one file.

Make a folder `providers`, inside it a folder `neon`, and inside that `neonDatabaseProvider.ts`:

```
// The ONE file that knows how to reach the database.
// A provider wraps something outside our app: here, Neon.
import { neon } from "@neondatabase/serverless";
import { drizzle, type NeonHttpDatabase } from "drizzle-orm/neon-http";

export class NeonDatabaseProvider {
  // One copy for the whole app, made the first time somebody asks.
  private static instance: NeonDatabaseProvider;
  private db: NeonHttpDatabase | undefined;

  public static getInstance(): NeonDatabaseProvider {
    if (!NeonDatabaseProvider.instance) {
      NeonDatabaseProvider.instance = new NeonDatabaseProvider();
    }
    return NeonDatabaseProvider.instance;
  }

  public getDb(): NeonHttpDatabase {
    // ...you write this part
  }
}
```

THE CLASS, IN THREE LINES YOU COPY

Wednesday's frame: `instance` is `static`, so there's one of it for the whole app. `getInstance()` makes it the first time anyone asks and hands back the same one every time after. So every route that says `NeonDatabaseProvider.getInstance()` gets the **same** provider, and the app connects once instead of once per request. You'll write the same three lines for the service in Step 6.

Now write `getDb()`. The first time it's called (`this.db` is still `undefined`):

1. Read `process.env.NEON_DATABASE_URL`.
2. If it's missing, `throw new Error(...)` with a sentence that says exactly what to do: `"NEON_DATABASE_URL is not set. Copy .env.example to .env.local and fill it in."` Failing loudly here, in plain words, beats a confusing crash somewhere else later.
3. Otherwise connect: `this.db = drizzle(neon(url));`

Every time, at the end: `return this.db;`

NOTHING TO TEST YET

Nothing calls the provider until Step 6, so there's nothing to see. VS Code shouldn't show any red underlines in the file.

Stuck? Code C3: the Neon provider

ASK YOUR CLAUDE

> *"Explain this TypeScript class to a beginner: [paste your provider]. What does static mean on the instance field, what does private mean, and why would getInstance() hand back the same object every time? What's this pattern called?"*

> *"Why is it better for the provider to throw a clear error when NEON_DATABASE_URL is missing, instead of letting neon(undefined) fail somewhere later?"*

6 The item service, and the list of coffees

The **service** is the one file that asks the database about items. Every query lives here, and so does every rule about the store. Routes call the service and never write a query themselves.

6a. The service

Make a folder `services` next to `providers`, and in it `itemService.ts`. Start with the imports, and the store's first rule: this screen is for **coffees** only, and coffee is category 1 in `store.sql`.

```
// The ONE file that knows how to ask the database about items.
// Routes call these methods. Routes never write a query themselves.
import { eq } from "drizzle-orm";
import { NeonDatabaseProvider } from "../providers/neon/neonDatabaseProvider";
import { items } from "../drizzle/schema";

// Coffee is category 1 in store.sql. This app only ever works on coffees,
// so every query below uses this rule, written once.
const coffeeCategoryId = 1;
const isCoffee = eq(items.categoryId, coffeeCategoryId);
```

`eq(items.categoryId, 1)` is Drizzle for the SQL `category_id = 1`. You're making it once, at the top, so every method below can use the same rule.

Then the class: the same three "one shared copy" lines as the provider (with `ItemService` instead of `NeonDatabaseProvider`), a small private helper that asks the provider for the connection, and your first query:

```
export class ItemService {
  // ...instance and getInstance(), like the provider

  private getDb() {
    return NeonDatabaseProvider.getInstance().getDb();
  }

  public async getCoffees() {
    return this.getDb().select().from(items).where(isCoffee).orderBy(items.id);
  }
}
```

Read that last line as SQL: `SELECT * FROM items WHERE category_id = 1 ORDER BY id`.

6b. The route calls it

In `app/api/coffees/route.ts`, import the service at the top. Count the folders: from `app/api/coffees/` it's three `../` back to the project folder:

```
import { ItemService } from "../../../services/itemService";
```

Then replace the `TODO (Step 6)` line and the `return Response.json([]);` under it with two lines: ask the service, send what it gives you. Leave the Steps 11 and 12 `TODO` where it is.

```
const coffees = await ItemService.getInstance().getCoffees();
return Response.json(coffees);
```

DON'T FORGET `AWAIT`

Talking to the database takes time, so `getCoffees()` gives back a **Promise** of rows, not the rows. Without `await`, `Response.json` gets the Promise, and a Promise turned into JSON is `{}`. The page then says "No coffees found." and nothing errors.

CHECK IT

Start the server again (`npm run dev`). Next only reads `.env.local` when it starts.

DO	EXPECT	YOU SEE
Open `localhost:3000/api/coffees`	**200**	8 coffees. `"price": 2.5` with **no quotes**: a number. Gingerbread Latte has `"isAvailable": false`.
Open `localhost:3000`	**page**	8 cards, Espresso to Gingerbread Latte, which says **Sold out**. No Cold Brew, no Chai Latte: they aren't coffees.

Error overlay: `coffee.price.toFixed is not a function`? The price came back as text. You skipped the edit in Step 4b.

500 and `relation "items" does not exist` in the terminal? The string points at `neondb`. Fix `.env.local` and restart.

Stuck? Code C4: the first service · C5: the list route calling it

ASK YOUR CLAUDE

> *"Here's my Drizzle query: this.getDb().select().from(items).where(isCoffee).orderBy(items.id), where isCoffee is eq(items.categoryId, 1). Write the SQL it turns into, and explain each piece."*

> *"What's the difference between returning a Promise and returning the value inside it? What happens if I pass a Promise to Response.json without await?"*

7 One coffee

The edit page fills its form from `GET /api/coffees/3`. Two parts, the same as Step 6: a method on the service, then the route calls it.

7a. `getCoffee(id)` in the service

This query needs **two** rules at once: the id matches, **and** it's a coffee. Drizzle's `and(...)` joins them, like `AND` in SQL. Add `and` to the import from `drizzle-orm`:

```
import { and, eq } from "drizzle-orm";
```

Then add the method inside the class:

```
// Gives back undefined when there's no such coffee.
public async getCoffee(id: number) {
  const [coffee] = await this.getDb()
    .select()
    .from(items)
    .where(and(eq(items.id, id), isCoffee));
  return coffee;
}
```

A `select` always gives back a list, even when it can only find one row. `const [coffee] = ...` takes the first thing in the list. If the list is empty, `coffee` is `undefined`.

7b. The route

In `app/api/coffees/[id]/route.ts`, import the service. This file is one folder deeper, so it's **four** `../`. In `GET`, replace the `TODO` and the 501: ask the service for `Number(id)` (the id in the address is text), answer **404** with `{"error": "No coffee with id ..."}` if it gave back nothing, otherwise send the coffee.

CHECK IT

DO	EXPECT	YOU SEE
Open `/api/coffees/3`	**200**	`{"id": 3, "name": "Latte", ..., "price": 4, ...}`
Open `/api/coffees/8`	**200**	Gingerbread Latte. Sold out, but it's still a coffee.
Open `/api/coffees/13`	**404**	`{"error": "No coffee with id 13"}`. Item 13 is Cold Brew. It's in the table, but it's not a coffee, and `isCoffee` says so.
Open `/api/coffees/99`	**404**	`{"error": "No coffee with id 99"}`
On the list, click **Edit** on Latte	**page**	The form, filled in: Latte, "Espresso and steamed milk.", 4. **Save** says "Not built yet. That's Step 9."

Look at `/api/coffees/13` again. You didn't write anything about Cold Brew. The rule you wrote once in Step 6 kept it out of the list **and** out of this route. That's the point of the service.

Stuck? Code C6: getCoffee and the one-coffee route

ASK YOUR CLAUDE

> *"In TypeScript, what does const [coffee] = await someList do? What is coffee if the list is empty? Why would a database select give back a list even when I'm looking up one id?"*

8 Create: a new coffee

8a. The shape of a coffee coming in

Adding and changing a coffee both need the same four things from the route. Give that shape a name, at the top of the service under `isCoffee`, and `export` it:

```
// What a route hands over to add or change a coffee. The route checks it first.
export type CoffeeInput = {
  name: string;
  description: string | null;
  price: number;
  isAvailable: boolean;
};
```

8b. `createCoffee(input)`

```
// .returning() hands back the row Postgres saved, new id included.
// categoryId is added here, so a new coffee is always a coffee.
public async createCoffee(input: CoffeeInput) {
  const [coffee] = await this.getDb()
    .insert(items)
    .values({ ...input, categoryId: coffeeCategoryId })
    .returning();
  return coffee;
}
```

Two things worth noticing. You don't send an `id`: the table's `serial` column picks the next one. And without `.returning()` an insert gives back nothing, so you'd never learn the id Postgres chose. `{ ...input, categoryId: coffeeCategoryId }` means "everything in `input`, plus `categoryId`." The route never chooses the category; the service does, using the same number as the rule.

8c. The route

In `POST` in `app/api/coffees/route.ts`, replace the `TODO` and the 501. Build the input from the body that was already checked, and answer **201** with the new coffee:

- `description: body.description || null`. An empty box (or no description at all) becomes `NULL`, which is how `store.sql` says "nobody wrote one."
- `isAvailable: true`. A new coffee starts on sale.

CHECK IT IN THE BROWSER

DO	EXPECT	YOU SEE
+ New coffee: `Cortado`, `Equal parts espresso and warm milk.`, `3.75`, Save	**201**	Back on the list: 9 cards, Cortado on the end.
Open `/api/coffees`	**200**	Cortado is there with `"id": 31` (if your store was fresh; the id is just the next number).
+ New coffee with an empty name, Save	**400**	The form shows **name is required**. That's the route's check, shown by the page.

CHECK IT IN POSTMAN

Postman desktop app, **Body** tab, **raw**, **JSON**. Save the requests in a **Coffee bar** collection; you'll re-run them.

SEND	BODY	EXPECT	BODY BACK
`POST /api/coffees`	`{"name": "Macchiato", "price": 3.25}`	**201**	Macchiato, `"description": null`, `"isAvailable": true`, the next id
`POST /api/coffees`	`{"name": "Macchiato", "price": "3.25"}`	**400**	`{"error": "price must be a number above 0"}`
`POST /api/coffees`	`null`	**400**	`{"error": "The body must be a JSON object with name, description and price"}`

NOW THE MOMENT THE WHOLE WEEK WAS FOR

Stop the server (`Ctrl` + `C`), start it again, and refresh the list. **Cortado and Macchiato are still there.** Last week a restart wiped every pet you added. Now the data lives in Postgres, not in your server's memory.

Prove it from the other side: Neon SQL Editor, `store`, `SELECT * FROM items WHERE category_id = 1;`. Ten rows, and yours are on the end.

Stuck? Code C7: CoffeeInput, createCoffee and POST

ASK YOUR CLAUDE

> *"In Drizzle, what does .returning() do on an insert, and what would I get back without it? Where does the new row's id come from when I never send one?"*

> *"What does { ...input, categoryId: 1 } do in JavaScript? Show me the object it makes if input is { name: 'Cortado', price: 3.75 }."*

9 Edit: change a coffee

The edit form sends the **whole** coffee back with `PUT`: every field, even the ones you didn't touch. (PUT means "here's the whole new thing, replace the old one," the same as week 11.) The route already checks all four fields.

9a. `updateCoffee(id, input)`

Write it yourself. It's `getCoffee` and `createCoffee` put together:

- `.update(items)` instead of `.insert(items)`
- `.set(input)`: the new values for the columns
- the **same** `.where(...)` as `getCoffee`, so only a coffee with that id changes
- `.returning()`, and `const [coffee] = ...`, and return it. `undefined` means no coffee had that id.

THE `WHERE` IS NOT OPTIONAL

An `update` with no `where` changes **every row in the table**. Every item in the store would get the same name and price, and there's no undo. The `isCoffee` rule also means this route can't touch a sandwich, even if someone sends its id.

9b. The route

In `PUT`, replace the `TODO` and the 501. Call `updateCoffee(Number(id), {...})` with the four fields (`description: body.description || null` again), answer **404** if it gave back nothing, otherwise send the changed coffee.

CHECK IT

DO	EXPECT	YOU SEE
Edit Gingerbread Latte: tick **On sale**, price `5`, Save	**200**	Back on the list: no **Sold out** tag, $5.00.

Edit Macchiato: description `A shot with a spot of foam.` , Save	**200**	The description shows on its card.
Postman `PUT /api/coffees/8` , body `{"isAvailable": false}`	**400**	`{"error": "name is required"}` . PUT wants the whole coffee.
Postman `PUT /api/coffees/13` , body `{"name": "Cold Brew", "description": "", "price": 1, "isAvailable": true}`	**404**	`{"error": "No coffee with id 13"}` . Cold Brew's price is safe.

Now ask the database about the Gingerbread Latte's **old** sales: `SELECT unit_price, count(*) FROM receipt_items WHERE item_id = 8 GROUP BY unit_price;` . They still say `4.75` . You changed today's price in `items` ; every receipt kept the price paid that day, in its own column. That's Tuesday's design doing its job.

Stuck? Code C8: updateCoffee and PUT

ASK YOUR CLAUDE

> *"What happens if I run a SQL UPDATE with no WHERE clause? Show me with UPDATE items SET price = 5. How would I protect against that in code?"*

> *"My store keeps today's price in items.price and the price actually paid in receipt_items.unit_price. Why keep it twice? What would go wrong with old receipts if they only pointed at items.price?"*

10 Delete, and Tuesday's foreign key

10a. The obvious version

Add `deleteCoffee(id)` to the service. A delete has nothing worth giving back, so it doesn't need `.returning()` :

```
public async deleteCoffee(id: number) {
  await this.getDb().delete(items).where(and(eq(items.id, id), isCoffee));
}
```

Same warning as Step 9, louder: a `delete` with no `where` empties the table.

In the route's `DELETE`, replace the `TODO` and the 501 with three moves: ask `getCoffee` first and answer **404** if there's no such coffee; then `deleteCoffee`; then answer **204** ("done, nothing to send back"). A 204 has no body at all:

```
return new Response(null, { status: 204 });
```

Since you call the service twice, keep it in a variable first: `const service = ItemService.getInstance();`.

CHECK IT, AND BREAK IT

DO	EXPECT	YOU SEE
Delete on Macchiato, OK	**204**	It's gone from the list. Refresh: still gone.
Postman `DELETE /api/coffees/` with Macchiato's id, again	**404**	`{"error": "No coffee with id ..."}`
Delete on Latte, OK	**500**	The page says **"The API answered 500. Check the terminal."**

Check the terminal:

```
× Error: Failed query: delete from "items" where ("items"."id" = $1 and "items"."category_id" = $2)
params: 3,1
  ...
  [cause]: Error [NeonDbError]: update or delete on table "items" violates foreign key constraint
"receipt_items_item_id_fkey" on table "receipt_items"
  ...
    detail: 'Key (id)=(3) is still referenced from table "receipt_items".',
```

That's Tuesday. `receipt_items.item_id  REFERENCES items (id)`, and thousands of old receipts have a Latte on them. If Postgres let the Latte go, those receipts would point at nothing. So it refused, and nothing was deleted. Refresh: the Latte is still there.

10b. Whose fault is that 500?

Remember week 11's rule: **4xx is the client's fault, 500 is ours.** This isn't a bug in your code, and the request wasn't bad either: "delete the Latte" is a perfectly sensible thing to ask. The **data** says no. HTTP has a status for exactly that: **409 Conflict**, "your

request is fine, but it conflicts with the state of things." And the owner deserves a sentence that says what to do instead.

The rule "a coffee that's been sold can't be deleted" is a rule about the store, so it belongs in the **service**. Add `receiptItems` to the schema import (`import { items, receiptItems } from "../drizzle/schema";`), then:

```
// Tuesday's foreign key: receipt_items points at items, so Postgres refuses
// to delete a coffee that's on an old receipt. Asking first lets the route
// say why, instead of crashing.
public async hasBeenSold(id: number) {
  const [line] = await this.getDb()
    .select()
    .from(receiptItems)
    .where(eq(receiptItems.itemId, id))
    .limit(1);
  return line !== undefined;
}
```

`.limit(1)` stops at the first receipt line it finds. You only need to know if there's **one**, not count all 26,000.

In the route, between the 404 check and the delete: if `await service.hasBeenSold(coffee.id)` , answer **409** with `{"error": "Latte is on old receipts, so it can't be deleted. Mark it sold out instead."}` (use `coffee.name` , not the word Latte).

CHECK IT

DO	EXPECT	YOU SEE
Delete on Latte, OK	**409**	Above the list, in red: **Latte is on old receipts, so it can't be deleted. Mark it sold out instead.**
Make a new coffee called `Test` , then **Delete** it	**204**	Gone. Nobody ever bought it, so there's nothing to protect.
Postman `DELETE /api/coffees/13`	**404**	Cold Brew isn't a coffee, so as far as this route knows, it doesn't exist.

Stuck? Code C9: deleteCoffee, hasBeenSold and DELETE

ASK YOUR CLAUDE

> *"What is a foreign key in Postgres, and why did DELETE FROM items WHERE id = 3 fail with 'violates foreign key constraint'? What are the options when you want to remove something that old records point at?"*

> *"Which status code fits each of these, and why: deleting a coffee that doesn't exist, deleting a coffee that's on old receipts, sending a price as text, and a bug in my own code."*

> *"A lot of real apps never really delete rows. They mark them as deleted or inactive instead. Why might a store want that?"*

11 Search by name: `?search=`

Type `lat` in the search box and press **Search**. The page shows **Asked: /api/coffees?search=lat**, and you still get every coffee. The page sent the search; the route isn't listening yet.

11a. Watch the SQL first

This step is about what your code sends to Postgres, so make it visible. In the provider, change the connect line to turn on Drizzle's logger:

```
// logger: true prints every query and its params in the terminal (Step 11).
this.db = drizzle(neon(url), { logger: true });
```

Restart, refresh the list, and look at the terminal. Every query is now printed, with its values listed separately at the end:

```
Query: select "id", "category_id", "name", ... from "items" where "items"."category_id" = $1 order by
"items"."id" -- params: [1]
```

See the `$1`? Drizzle never glues your values into the SQL text. It sends the SQL with numbered blanks, and the values separately, as **parameters**. Keep that in mind for 11d.

11b. The route reads the parameter

Everything after the `?` in an address is the **query string**: named values, joined with `&`. In a route, read them like this (you saw it in week 11's "Going further"):

```
// ?search=lat -- text, and it can be missing.
const query = new URL(request.url).searchParams;
const search = query.get("search") ?? "";
```

`query.get("search")` gives `"lat"`, or `null` if there's no `search` in the address. `?? ""` turns that `null` into an empty string. Pass `search` to `getCoffees(search)`. You can delete the Steps 11 and 12 `TODO` now.

11c. The service uses it

Give `getCoffees` a `search: string` input, add `ilike` to the `drizzle-orm` import, and add a second rule next to `isCoffee`:

```
.where(and(isCoffee, ilike(items.name, `%${search}%`)))
```

That's the SQL `name ILIKE '%lat%'`. `LIKE` matches a pattern, where `%` means "any characters, or none," so `%lat%` is "`lat` anywhere in the name." The **I** in `ILIKE` means it ignores capital letters. And an empty search, `%%`, matches every name, which is why "no search" still lists everything.

11d. Experiment

Try each of these, in the search box **or** typed straight into the browser's address bar. **Predict the result before you press Enter**, then read the `Query:` line in the terminal.

CHECK IT

SEARCH FOR	YOU GET	WHY
`latte`	Latte, Gingerbread Latte	Not Chai Latte or Iced Latte: they're not coffees. Two rules, both in the `where`.
`LATTE`	The same two	`ilike` ignores case. Try `like` instead of `ilike` in the service: now `LATTE` finds nothing. Put `ilike` back.
`lat`	Latte, Flat White, Gingerbread Latte	Did you predict Flat White? F-**lat**. "Anywhere in the name" means anywhere.
`tea`	Nothing	There are teas in the store. None of them are coffees.
(empty)	Every coffee	The terminal shows `params: [1, "%%"]`.
`' OR 1=1 --`	Nothing	See below.

THE TEACHING POINT: PARAMETERS, NOT PASTED TEXT

`' OR 1=1 --` is the oldest attack on a database. If code built the SQL by pasting text together, the query would become `... WHERE name ILIKE '%' OR 1=1 --%'`: the quote ends the text early, `OR 1=1` is true for every row, and `--` turns the rest into a comment. On a login form, that's how people get in without a password. It's called **SQL injection**.

Look at your terminal:

```
... where ("items"."category_id" = $1 and "items"."name" ilike $2) ... -- params: [1, "%' OR 1=1 --%"]
```

The SQL is exactly the same shape as for `latte`. The attack arrived as a **value** in `$2`, and Postgres went looking for a coffee with that odd name. There isn't one. Because every query goes through Drizzle, and every query lives in the service, there's no place in this app where someone's text gets pasted into SQL.

Stuck? Code C10: search, in the service and the route

ASK YOUR CLAUDE

> *"What is SQL injection? Show me how "SELECT * FROM items WHERE name LIKE '%" + search + "%'" goes wrong when search is ' OR 1=1 --, and how a parameterized query with $1 stops it."*

> *"In a URL like /api/coffees?search=lat&maxPrice=4, what's the query string? What does new URL(request.url).searchParams.get('search') give back when search is missing, and why is it null and not undefined?"*

> *"In SQL, what's the difference between LIKE and ILIKE, and what do % and _ mean inside the pattern?"*

12 A second parameter: `?maxPrice=`

The page has a second box, **Max price**, and sends it as `&maxPrice=4`. This one is different from `search` in two ways: it has to be a **number**, and when it's missing there should be **no price rule at all** (not "a price of 0").

12a. The route

Under the `search` line, read `maxPrice`. If it's in the address, turn it into a number and check it, the week 11 way: `Number("abc")` is `NaN` ("not a number"), and a price of 0 or less makes no sense, so either one is a **400**. If it's not in the address, leave it `undefined`:

```
let maxPrice: number | undefined;
const maxPriceText = query.get("maxPrice");
if (maxPriceText !== null) {
  maxPrice = Number(maxPriceText);
  if (Number.isNaN(maxPrice) || maxPrice <= 0) {
    return Response.json({ error: "maxPrice must be a number above 0" }, { status: 400 });
  }
}
```

Then pass both: `getCoffees(search, maxPrice)`.

12b. The service

Give `getCoffees` a second input, `maxPrice?: number` (the `?` means it can be left out), and add `lte` ("less than or equal") to the `drizzle-orm` import. Then

a third rule in the `and(...)`:

```
maxPrice === undefined ? undefined : lte(items.price, maxPrice)
```

Here's the trick: Drizzle's `and()` **skips** anything that's `undefined`. So with no `maxPrice`, that rule simply isn't part of the `where`. With one, it's `price <= 4`.

CHECK IT: PREDICT, THEN PRESS ENTER

These assume the coffees from Steps 8–10: the original 8 (Gingerbread now $5.00) plus Cortado. Watch the `params:` in the terminal each time; count how many there are.

ADDRESS	EXPECT	YOU GET
`/api/coffees?maxPrice=4`	**200**	Espresso, Americano, Latte, Cappuccino, Cortado. Latte is exactly $4.00, and `lte` includes it.
`/api/coffees?search=latte&maxPrice=4`	**200**	Latte only. Three rules, joined with `and`: `params: [1, "%latte%", 4]`
`/api/coffees?maxPrice=4.25`	**200**	The five, plus Flat White
`/api/coffees?maxPrice=abc`	**400**	`{"error": "maxPrice must be a number above 0"}`, and no `Query:` line. The route stopped it before the service was asked.
`/api/coffees?maxPrice=0`	**400**	The same message
`/api/coffees?maxprice=4`	**200**	Every coffee! Names in the query string are case-sensitive. `maxprice` isn't `maxPrice`, so the route never saw it, and unknown parameters are simply ignored.
`/api/coffees?search=mocha&search=latte`	**200**	Mocha. `get()` gives back the **first** one. (`getAll()` would give you both.)

Then use the boxes on the page: `lat` and `4.5` gives Latte and Flat White, and the page shows the exact address it asked for.

Stuck? Code C11: maxPrice, in the service and the route

ASK YOUR CLAUDE

> *"In TypeScript, what's the difference between a parameter written maxPrice?: number and one written maxPrice: number | undefined? When would I use each?"*

> *"Why does Number('abc') give NaN, and why can't I check for it with maxPrice === NaN? What does Number.isNaN do differently?"*

> *"Here's my getCoffees method: [paste it]. Without changing it, tell me the exact SQL WHERE clause and params it sends for: no search and no maxPrice; search 'lat' only; maxPrice 4 only; both."*

What "done" looks like

- The list shows every coffee from your `store` database, with a **Sold out** tag where `is_available` is false, and nothing that isn't a coffee.
- **+ New coffee**, **Edit** and **Delete** all work, and what they change is still there after you restart the server.
- Deleting a coffee that's on old receipts shows a 409 message that says what to do instead. Deleting one nobody has bought works.
- `?search=` and `?maxPrice=` work alone and together, and a bad `maxPrice` is a 400.
- **One** file connects (`providers/neon/neonDatabaseProvider.ts`). **One** file writes queries (`services/itemService.ts`). The route files import neither `drizzle-orm` nor the schema nor the provider, only the service.
- The "is a coffee" rule is written **once**, and all five methods that touch `items` use it.
- `git status` doesn't list `.env.local` .
- `npm run build` finishes with no errors.

Going further

Finished early? Each of these is a real hole or a real feature. Try to explain what's going on **before** you fix it, and keep the rule: checks on the request in the route, queries and store rules in the service.

1. **The id that isn't a number.** Open `/api/coffees/abc` . Last week the same address on the pets API was a 404. Now it's a 500, and the terminal says `invalid input syntax for type integer: "NaN"` . Why did it change? Make it a 404 again, in the route, before the service is asked. Do all three verbs in that file need it?
2. **A name that's taken.** Add a coffee called `Cortado` a second time. 500, and the terminal says `duplicate key value violates unique constraint "items_name_key"` . Now try a coffee called `Lemonade` . Why does that fail too, when there's no coffee called Lemonade? (Hint: Tuesday made `name` unique across every item, not just coffees.) Turn it into a 409 that says the name is taken, the way you did for delete.
3. **A very expensive coffee.** Save a coffee with price `10000` . 500: `numeric field overflow` . Look at `price numeric(6,2)` in `db/0001_store.sql` and work out the biggest price it can hold. Then try

`4.999` and read what Postgres saved. Add a check to the route so neither surprise reaches the database.

4. **The safety net.** Week 11 Step 10 wrapped POST and PUT in an outer `try` / `catch` that answers `{"error": "Something went wrong on our side"}` and logs the real error with `console.error`. It said "next week, talking to a database" is where things really fail. Add it to every route that calls the service. Test it by breaking the password in `.env.local` and restarting. Then remove the line from `.env.local` completely and restart: whose error message do you see in the terminal now?
5. **Wildcards in the search box.** Search for `%`, then `_`. Both give you every coffee. Why? (Hint: what do those two mean inside a `LIKE` pattern?) Is that a security hole, or just a surprise? Ask your Claude how to make them match the real characters.
6. **Your own parameter.** Add `?sort=price` that orders the list cheapest first. Drizzle's `asc()` and `desc()` go inside `.orderBy(...)`. What should the route do with `?sort=banana`? Then add a **Sort by price** box to the page and send it. (Yes, this one changes the page.)

Things that will bite you

- **The wrong database.** Neon's Connect box starts on `neondb`. The string connects fine, so the pull says `1 tables fetched` or `0`, and a route says `relation "items" does not exist`. Check the end of the string first.
- **After a wrong pull, delete `drizzle/`.** Pulling again rewrites `schema.ts`, but leaves the first pull's `0000_...sql` and `meta/` behind, still describing the wrong database.
- **Pulling again erases your edit.** `mode: "number"` is gone, and the page crashes with `toFixed is not a function`.
- **Restart after changing `.env.local`.** Next only reads it when it starts. drizzle-kit reads it every time you run it.
- **The pull needs Node 22 or newer.** Check with `node --version`.
- **Counting `../`.** Three from `app/api/coffees/route.ts`, four from `app/api/coffees/[id]/route.ts`, one from `services/itemService.ts`. A red underline on the import line is the tell; the error says `Module not found`.
- **A forgotten `await`** sends `{}`, and the list says "No coffees found." with no error anywhere.
- **A 500 with no message on the page** means something threw. The terminal has the reason, down to the line, usually with a `[cause]:` from Postgres. Read it before you change anything.
- **Your changes are real now.** Every create, edit and delete stays in your database, and the expected results in later steps assume you did the earlier ones. If your

store gets into a strange state, run `db/0001_store.sql` again in the SQL Editor on `store`; everything goes back to the way it started.

- **Postman on the web can't reach `localhost`.** Use the desktop app. Body type **raw**, **JSON**, and `"price": 3.25` (a number), not `"3.25"` (text).
- **Claude says Prisma, `pg`, or puts the query in the route.** Remind it: Drizzle on neon-http, every query in the service.
- **Port 3000 taken?** Next picks 3001 and says so in the terminal. Use whatever it says, or stop the other `npm run dev`.

Code reference

The code at the end of each step. Try the step first; come here when you're stuck or want to compare. Each block says which file it goes in and links back to its step. C12–C15 are the four finished files, complete.

C1 · The config and the script

↑ back to Step 3

`drizzle.config.ts`

```
// Settings for drizzle-kit, the command-line half of Drizzle.
// Next reads .env.local by itself. drizzle-kit doesn't, so dotenv loads it here.
import { defineConfig } from "drizzle-kit";
import dotenv from "dotenv";

dotenv.config({ path: ".env.local" });

export default defineConfig({
  dialect: "postgresql",             // which kind of database
  schema: "./drizzle/schema.ts",     // where our tables are described
  out: "./drizzle",                  // where drizzle-kit writes what it generates
  dbCredentials: { url: process.env.NEON_DATABASE_URL || "" },
});
```

`package.json (the "scripts" part)`

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "db:pull": "drizzle-kit pull"
},
```

C2 · The items table after the edit

↑ back to Step 4

`drizzle/schema.ts (only the items table; the pull wrote all ten)`

```
export const items = pgTable("items", {
    id: serial().primaryKey().notNull(),
    categoryId: integer("category_id").notNull(),
    name: text().notNull(),
    description: text(),
    // Postgres sends numeric back as text ("2.50") so it can't lose a penny.
    // mode: "number" tells Drizzle to turn it into a JavaScript number for us.
    price: numeric({ precision: 6, scale: 2, mode: "number" }).notNull(),
    isAvailable: boolean("is_available").default(true).notNull(),
    createdAt: timestamp("created_at", { withTimezone: true, mode: 'string' }).defaultNow().notNull(),
```

```
}, (table) => [
    foreignKey({
            columns: [table.categoryId],
            foreignColumns: [categories.id],
            name: "items_category_id_fkey"
        }),
    unique("items_name_key").on(table.name),
    check("items_price_check", sql`price >= (0)::numeric`),
]);
```

C3 · The Neon provider

↑ back to Step 5

providers/neon/neonDatabaseProvider.ts

```
// The ONE file that knows how to reach the database.
// A provider wraps something outside our app: here, Neon.
import { neon } from "@neondatabase/serverless";
import { drizzle, type NeonHttpDatabase } from "drizzle-orm/neon-http";

export class NeonDatabaseProvider {
  // One copy for the whole app, made the first time somebody asks.
  private static instance: NeonDatabaseProvider;
  private db: NeonHttpDatabase | undefined;

  public static getInstance(): NeonDatabaseProvider {
    if (!NeonDatabaseProvider.instance) {
      NeonDatabaseProvider.instance = new NeonDatabaseProvider();
    }
    return NeonDatabaseProvider.instance;
  }

  public getDb(): NeonHttpDatabase {
    if (!this.db) {
      // If the secret is missing, say so in plain words here,
      // instead of failing somewhere weird later.
      const url = process.env.NEON_DATABASE_URL;
      if (!url) {
        throw new Error(
          "NEON_DATABASE_URL is not set. Copy .env.example to .env.local and fill it in."
        );
      }
      this.db = drizzle(neon(url));
    }
    return this.db;
  }
}
```

C4 · The first service

↑ back to Step 6

services/itemService.ts

```
// The ONE file that knows how to ask the database about items.
// Routes call these methods. Routes never write a query themselves.
import { eq } from "drizzle-orm";
import { NeonDatabaseProvider } from "../providers/neon/neonDatabaseProvider";
import { items } from "../drizzle/schema";

// Coffee is category 1 in store.sql. This app only ever works on coffees,
// so every query below uses this rule, written once.
const coffeeCategoryId = 1;
const isCoffee = eq(items.categoryId, coffeeCategoryId);

export class ItemService {
  private static instance: ItemService;

  public static getInstance(): ItemService {
    if (!ItemService.instance) {
      ItemService.instance = new ItemService();
    }
    return ItemService.instance;
  }

  private getDb() {
    return NeonDatabaseProvider.getInstance().getDb();
  }

  public async getCoffees() {
    return this.getDb().select().from(items).where(isCoffee).orderBy(items.id);
  }
}
```

C5 · The list route calling it

↑ back to Step 6

app/api/coffees/route.ts (the top of the file; POST is unchanged until Step 8)

```
// /api/coffees -- every coffee, and adding a new one.
// No connection, no query, no import from drizzle. Check the request, ask
// the service, pick the status code.
import { ItemService } from "../../../services/itemService";

export async function GET(request: Request) {
  // TODO (Steps 11 and 12): read ?search= and ?maxPrice= from the address.
  const coffees = await ItemService.getInstance().getCoffees();
  return Response.json(coffees);
}
```

C6 · getCoffee and the one-coffee route

↑ back to Step 7

services/itemService.ts (the import, and a new method inside the class)

```
import { and, eq } from "drizzle-orm";

  // Gives back undefined when there's no such coffee.
  public async getCoffee(id: number) {
    const [coffee] = await this.getDb()
      .select()
      .from(items)
      .where(and(eq(items.id, id), isCoffee));
    return coffee;
  }
```

app/api/coffees/[id]/route.ts (the import, and GET)

```
import { ItemService } from "../../../../services/itemService";

export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const coffee = await ItemService.getInstance().getCoffee(Number(id));
  if (!coffee) {
    return Response.json({ error: `No coffee with id ${id}` }, { status: 404 });
  }
  return Response.json(coffee);
}
```

C7 · CoffeeInput, createCoffee and POST

↑ back to Step 8

services/itemService.ts (the type under isCoffee, and a new method inside the class)

```
// What a route hands over to add or change a coffee. The route checks it first.
export type CoffeeInput = {
  name: string;
  description: string | null;
  price: number;
  isAvailable: boolean;
};

  // .returning() hands back the row Postgres saved, new id included.
  // categoryId is added here, so a new coffee is always a coffee.
  public async createCoffee(input: CoffeeInput) {
    const [coffee] = await this.getDb()
      .insert(items)
      .values({ ...input, categoryId: coffeeCategoryId })
      .returning();
    return coffee;
  }
```

app/api/coffees/route.ts (POST)

```
export async function POST(request: Request) {
  let body;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  if (body === null || typeof body !== "object") {
    return Response.json(
      { error: "The body must be a JSON object with name, description and price" },
      { status: 400 }
    );
  }
  if (typeof body.name !== "string" || body.name === "") {
    return Response.json({ error: "name is required" }, { status: 400 });
  }
  // description is optional: leave it out, or send "", and the coffee has none.
  if (body.description !== undefined && typeof body.description !== "string") {
    return Response.json({ error: "description must be text" }, { status: 400 });
  }
  if (typeof body.price !== "number" || body.price <= 0) {
    return Response.json({ error: "price must be a number above 0" }, { status: 400 });
  }

  const coffee = await ItemService.getInstance().createCoffee({
    name: body.name,
    description: body.description || null, // no description is NULL, like store.sql
    price: body.price,
    isAvailable: true, // a new coffee starts on sale
  });
  return Response.json(coffee, { status: 201 });
}
```

C8 · updateCoffee and PUT

↑ back to Step 9

services/itemService.ts (a new method inside the class)

```
  // Gives back undefined when there's no such coffee.
  public async updateCoffee(id: number, input: CoffeeInput) {
    const [coffee] = await this.getDb()
      .update(items)
      .set(input)
      .where(and(eq(items.id, id), isCoffee))
      .returning();
    return coffee;
  }
```

app/api/coffees/[id]/route.ts (PUT)

```
export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;

  let body;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  // PUT replaces the whole coffee, so every field has to be there.
  if (body === null || typeof body !== "object") {
    return Response.json(
      { error: "The body must be a JSON object with name, description, price and isAvailable" },
      { status: 400 }
    );
  }
  if (typeof body.name !== "string" || body.name === "") {
    return Response.json({ error: "name is required" }, { status: 400 });
  }
  if (typeof body.description !== "string") {
    return Response.json({ error: "description must be text (send \"\" for none)" }, { status: 400 });
  }
  if (typeof body.price !== "number" || body.price <= 0) {
    return Response.json({ error: "price must be a number above 0" }, { status: 400 });
  }
  if (typeof body.isAvailable !== "boolean") {
    return Response.json({ error: "isAvailable must be true or false" }, { status: 400 });
  }

  const coffee = await ItemService.getInstance().updateCoffee(Number(id), {
    name: body.name,
    description: body.description || null, // no description is NULL, like store.sql
    price: body.price,
    isAvailable: body.isAvailable,
  });
  if (!coffee) {
    return Response.json({ error: `No coffee with id ${id}` }, { status: 404 });
  }
  return Response.json(coffee);
}
```

C9 · deleteCoffee, hasBeenSold and DELETE

↑ back to Step 10

services/itemService.ts (the schema import, and two new methods inside the class)

```
import { items, receiptItems } from "../drizzle/schema";

  public async deleteCoffee(id: number) {
    await this.getDb().delete(items).where(and(eq(items.id, id), isCoffee));
  }

  // Tuesday's foreign key: receipt_items points at items, so Postgres refuses
  // to delete a coffee that's on an old receipt. Asking first lets the route
  // say why, instead of crashing.
  public async hasBeenSold(id: number) {
    const [line] = await this.getDb()
      .select()
      .from(receiptItems)
      .where(eq(receiptItems.itemId, id))
      .limit(1);
    return line !== undefined;
  }
```

app/api/coffees/[id]/route.ts (DELETE, with 10b's 409)

```
export async function DELETE(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const service = ItemService.getInstance();

  const coffee = await service.getCoffee(Number(id));
  if (!coffee) {
    return Response.json({ error: `No coffee with id ${id}` }, { status: 404 });
  }
  // 409 Conflict: the request is fine, but the data says no.
  if (await service.hasBeenSold(coffee.id)) {
    return Response.json(
      { error: `${coffee.name} is on old receipts, so it can't be deleted. Mark it sold out instead.` },
      { status: 409 }
    );
  }

  await service.deleteCoffee(coffee.id);
  return new Response(null, { status: 204 });
}
```

C10 · Search, in the service and the route

↑ back to Step 11

providers/neon/neonDatabaseProvider.ts (the connect line)

```
      // logger: true prints every query and its params in the terminal (Step 11).
      this.db = drizzle(neon(url), { logger: true });
```

services/itemService.ts (the import, and getCoffees replaced)

```
import { and, eq, ilike } from "drizzle-orm";

  public async getCoffees(search: string) {
    return this.getDb()
      .select()
      .from(items)
      .where(and(isCoffee, ilike(items.name, `%${search}%`)))
      .orderBy(items.id);
  }
```

app/api/coffees/route.ts (GET)

```
export async function GET(request: Request) {
  // ?search=lat -- text, and it can be missing.
  const query = new URL(request.url).searchParams;
  const search = query.get("search") ?? "";

  const coffees = await ItemService.getInstance().getCoffees(search);
  return Response.json(coffees);
}
```

C11 · maxPrice, in the service and the route

↑ back to Step 12

services/itemService.ts (the import, and getCoffees replaced)

```
import { and, eq, ilike, lte } from "drizzle-orm";

  // search "" matches every name. maxPrice is optional: and() skips anything
  // that's undefined, so with no maxPrice there's simply no price rule.
  public async getCoffees(search: string, maxPrice?: number) {
    return this.getDb()
      .select()
      .from(items)
      .where(
        and(
          isCoffee,
          ilike(items.name, `%${search}%`),
          maxPrice === undefined ? undefined : lte(items.price, maxPrice)
        )
      )
      .orderBy(items.id);
  }
```

app/api/coffees/route.ts (GET)

```
export async function GET(request: Request) {
  // ?search=lat&maxPrice=4 -- both are text, and both can be missing.
  const query = new URL(request.url).searchParams;
  const search = query.get("search") ?? "";
```

```
  let maxPrice: number | undefined;
  const maxPriceText = query.get("maxPrice");
  if (maxPriceText !== null) {
    maxPrice = Number(maxPriceText);
    if (Number.isNaN(maxPrice) || maxPrice <= 0) {
      return Response.json({ error: "maxPrice must be a number above 0" }, { status: 400 });
    }
  }

  const coffees = await ItemService.getInstance().getCoffees(search, maxPrice);
  return Response.json(coffees);
}
```

C12 · The finished service

↑ back to Step 12

services/itemService.ts

```
// The ONE file that knows how to ask the database about items.
// Routes call these methods. Routes never write a query themselves.
import { and, eq, ilike, lte } from "drizzle-orm";
import { NeonDatabaseProvider } from "../providers/neon/neonDatabaseProvider";
import { items, receiptItems } from "../drizzle/schema";

// Coffee is category 1 in store.sql. This app only ever works on coffees,
// so every query below uses this rule, written once.
const coffeeCategoryId = 1;
const isCoffee = eq(items.categoryId, coffeeCategoryId);

// What a route hands over to add or change a coffee. The route checks it first.
export type CoffeeInput = {
  name: string;
  description: string | null;
  price: number;
  isAvailable: boolean;
};

export class ItemService {
  private static instance: ItemService;

  public static getInstance(): ItemService {
    if (!ItemService.instance) {
      ItemService.instance = new ItemService();
    }
    return ItemService.instance;
  }

  private getDb() {
    return NeonDatabaseProvider.getInstance().getDb();
  }

  // search "" matches every name. maxPrice is optional: and() skips anything
  // that's undefined, so with no maxPrice there's simply no price rule.
  public async getCoffees(search: string, maxPrice?: number) {
```

```
  return this.getDb()
    .select()
    .from(items)
    .where(
      and(
        isCoffee,
        ilike(items.name, `%${search}%`),
        maxPrice === undefined ? undefined : lte(items.price, maxPrice)
      )
    )
    .orderBy(items.id);
}

// Gives back undefined when there's no such coffee.
public async getCoffee(id: number) {
  const [coffee] = await this.getDb()
    .select()
    .from(items)
    .where(and(eq(items.id, id), isCoffee));
  return coffee;
}

// .returning() hands back the row Postgres saved, new id included.
// categoryId is added here, so a new coffee is always a coffee.
public async createCoffee(input: CoffeeInput) {
  const [coffee] = await this.getDb()
    .insert(items)
    .values({ ...input, categoryId: coffeeCategoryId })
    .returning();
  return coffee;
}

// Gives back undefined when there's no such coffee.
public async updateCoffee(id: number, input: CoffeeInput) {
  const [coffee] = await this.getDb()
    .update(items)
    .set(input)
    .where(and(eq(items.id, id), isCoffee))
    .returning();
  return coffee;
}

public async deleteCoffee(id: number) {
  await this.getDb().delete(items).where(and(eq(items.id, id), isCoffee));
}

// Tuesday's foreign key: receipt_items points at items, so Postgres refuses
// to delete a coffee that's on an old receipt. Asking first lets the route
// say why, instead of crashing.
public async hasBeenSold(id: number) {
  const [line] = await this.getDb()
    .select()
    .from(receiptItems)
    .where(eq(receiptItems.itemId, id))
    .limit(1);
  return line !== undefined;
```

```
  }
}
```

C13 · The finished list route

↑ back to Step 12

app/api/coffees/route.ts

```
// /api/coffees -- every coffee, and adding a new one.
// No connection, no query, no import from drizzle. Check the request, ask
// the service, pick the status code.
import { ItemService } from "../../../services/itemService";

export async function GET(request: Request) {
  // ?search=lat&maxPrice=4 -- both are text, and both can be missing.
  const query = new URL(request.url).searchParams;
  const search = query.get("search") ?? "";

  let maxPrice: number | undefined;
  const maxPriceText = query.get("maxPrice");
  if (maxPriceText !== null) {
    maxPrice = Number(maxPriceText);
    if (Number.isNaN(maxPrice) || maxPrice <= 0) {
      return Response.json({ error: "maxPrice must be a number above 0" }, { status: 400 });
    }
  }

  const coffees = await ItemService.getInstance().getCoffees(search, maxPrice);
  return Response.json(coffees);
}

export async function POST(request: Request) {
  let body;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  if (body === null || typeof body !== "object") {
    return Response.json(
      { error: "The body must be a JSON object with name, description and price" },
      { status: 400 }
    );
  }
  if (typeof body.name !== "string" || body.name === "") {
    return Response.json({ error: "name is required" }, { status: 400 });
  }
  // description is optional: leave it out, or send "", and the coffee has none.
  if (body.description !== undefined && typeof body.description !== "string") {
    return Response.json({ error: "description must be text" }, { status: 400 });
  }
  if (typeof body.price !== "number" || body.price <= 0) {
    return Response.json({ error: "price must be a number above 0" }, { status: 400 });
```

```
  }

  const coffee = await ItemService.getInstance().createCoffee({
    name: body.name,
    description: body.description || null, // no description is NULL, like store.sql
    price: body.price,
    isAvailable: true, // a new coffee starts on sale
  });
  return Response.json(coffee, { status: 201 });
}
```

C14 · The finished one-coffee route

↑ back to Step 10

app/api/coffees/[id]/route.ts

```
// /api/coffees/3 -- one coffee: read it, change it, delete it.
// Same shape as the list route: ask the service, then decide the status code.
import { ItemService } from "../../../../services/itemService";

export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const coffee = await ItemService.getInstance().getCoffee(Number(id));
  if (!coffee) {
    return Response.json({ error: `No coffee with id ${id}` }, { status: 404 });
  }
  return Response.json(coffee);
}

export async function PUT(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;

  let body;
  try {
    body = await request.json();
  } catch {
    return Response.json({ error: "The body must be JSON" }, { status: 400 });
  }

  // PUT replaces the whole coffee, so every field has to be there.
  if (body === null || typeof body !== "object") {
    return Response.json(
      { error: "The body must be a JSON object with name, description, price and isAvailable" },
      { status: 400 }
    );
  }
  if (typeof body.name !== "string" || body.name === "") {
    return Response.json({ error: "name is required" }, { status: 400 });
```

```
  }
  if (typeof body.description !== "string") {
    return Response.json({ error: "description must be text (send \"\" for none)" }, { status: 400 });
  }
  if (typeof body.price !== "number" || body.price <= 0) {
    return Response.json({ error: "price must be a number above 0" }, { status: 400 });
  }
  if (typeof body.isAvailable !== "boolean") {
    return Response.json({ error: "isAvailable must be true or false" }, { status: 400 });
  }

  const coffee = await ItemService.getInstance().updateCoffee(Number(id), {
    name: body.name,
    description: body.description || null, // no description is NULL, like store.sql
    price: body.price,
    isAvailable: body.isAvailable,
  });
  if (!coffee) {
    return Response.json({ error: `No coffee with id ${id}` }, { status: 404 });
  }
  return Response.json(coffee);
}

export async function DELETE(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const service = ItemService.getInstance();

  const coffee = await service.getCoffee(Number(id));
  if (!coffee) {
    return Response.json({ error: `No coffee with id ${id}` }, { status: 404 });
  }
  // 409 Conflict: the request is fine, but the data says no.
  if (await service.hasBeenSold(coffee.id)) {
    return Response.json(
      { error: `${coffee.name} is on old receipts, so it can't be deleted. Mark it sold out instead.` },
      { status: 409 }
    );
  }

  await service.deleteCoffee(coffee.id);
  return new Response(null, { status: 204 });
}
```

C15 · The finished provider

↑ back to Step 11

providers/neon/neonDatabaseProvider.ts

```
// The ONE file that knows how to reach the database.
// A provider wraps something outside our app: here, Neon.
import { neon } from "@neondatabase/serverless";
```

```
import { drizzle, type NeonHttpDatabase } from "drizzle-orm/neon-http";

export class NeonDatabaseProvider {
  // One copy for the whole app, made the first time somebody asks.
  private static instance: NeonDatabaseProvider;
  private db: NeonHttpDatabase | undefined;

  public static getInstance(): NeonDatabaseProvider {
    if (!NeonDatabaseProvider.instance) {
      NeonDatabaseProvider.instance = new NeonDatabaseProvider();
    }
    return NeonDatabaseProvider.instance;
  }

  public getDb(): NeonHttpDatabase {
    if (!this.db) {
      // If the secret is missing, say so in plain words here,
      // instead of failing somewhere weird later.
      const url = process.env.NEON_DATABASE_URL;
      if (!url) {
        throw new Error(
          "NEON_DATABASE_URL is not set. Copy .env.example to .env.local and fill it in."
        );
      }
      // logger: true prints every query and its params in the terminal (Step 11).
      this.db = drizzle(neon(url), { logger: true });
    }
    return this.db;
  }
}
```
